Vulnerable Bank (VB) PCI Compliance Audit & Remediation Report

Prepared by: Charmaine Mangorima

Date: 22 January 2026

Target: Vulnerable Bank (VB)

Audit Focus: Cardholder Data Leaks

Compliance Standard: PCI DSS

1. Executive Summary

This report documents a security audit of the Vulnerable Bank (VB) API with a specific focus on **PCI DSS compliance and protection of cardholder data**.

The objective of the assessment was to identify API endpoints that expose sensitive payment card information and evaluate their compliance against **PCI DSS requirements**.

The assessment revealed multiple critical vulnerabilities, including exposure of full card numbers, CVVs, billing account numbers and payment references through unsecured API responses. These weaknesses represent direct violations of PCI DSS requirements related to encryption, tokenization, access control, and data minimization.

Practical remediation strategies, including **card tokenization, field-level encryption, access control enforcement and response filtering**, are proposed to align the API with PCI DSS compliance.

PCI DSS Requirements

1. Install and maintain network security controls
2. Apply secure configurations to all system components
3. Protect stored account data
4. Protect cardholder data with strong cryptography during transmission over open, public networks
5. Protect all systems and networks from malicious software
6. Develop and maintain secure systems and software

7. Restrict access to system components and cardholder data by business need to know
8. Identify users and authenticate access to system components
9. Restrict physical access to cardholder data
10. Log and monitor all access to system components and cardholder data
11. Test security of systems and network regularly
12. Support information security with organizational policies and programs

2. Scope of Assessment

- **In-Scope:** All API endpoints processing financial data and authentication routes.
- **Out-of-Scope:** Frontend UI and internal infrastructure not directly interacting with the API.

3. Methodology

Tools Used

- Postman
- Burpsuite
- VulnBank Swagger UI (https://vulnbank.org/api/docs/)

Testing Approach

1. Reviewed OpenAPI specification for sensitive data fields
2. Identified endpoints handling payment cards and user data
3. Tested endpoints via Postman and Burpsuite
4. Captured responses exposing cardholder data
5. Mapped findings to PCI DSS & OWASP API standards

4. Detailed Audit Findings & PCI Violations

I opened the API documentation on https://vulnbank.org/api/docs/ and identified Cardholder Data endpoints

virtual-cards Virtual card management endpoints

POST	/api/virtual-cards/create	Create a new virtual card
GET	/api/virtual-cards	Get user's virtual cards
POST	/api/virtual-cards/{card_id}/toggle-freeze	Toggle card freeze status
GET	/api/virtual-cards/{card_id}/transactions	Get card transactions
POST	/api/virtual-cards/{card_id}/update-limit	Update card limit

bill-payments Bill payment operations

GET	/api/bill-categories	Get bill categories
GET	/api/billers/by-category/{category_id}	Get billers by category
POST	/api/bill-payments/create	Create bill payment
GET	/api/bill-payments/history	Get payment history

Finding 1: Registration Endpoint Returns Debug Information

Endpoint: *POST /register*

Description

The registration endpoint returns a debug_data object containing internal system information.

Evidence

I registered 2 test users and the API response returns sensitive user information such as the raw username and password, account number and balance.

user1 registration

POST {{baseUrl}} /register Send

Docs Params Auth Headers (10) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit

Body 200 OK 391 ms 1.24 KB

{} JSON Preview Visualize


```
{
    "debug_data": {
        "account_number": "4875095390",
        "balance": 1000.0,
        "fields_registered": [
            "username",
            "password",
            "account_number"
        ],
        "is_admin": false,
        "raw_data": {
            "password": "password123",
            "username": "user1"
        },
        "registration_time": "2026-01-24 17:34:09.618391",
        "server_info": "PostmanRuntime/7.51.0",
        "user_id": 1460,
        "username": "user1"
```


user2 registration

POST {{baseUrl}} /register Send

Docs Params Authorization Headers (11) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
  "username": "user2",
  "password": "password123"
}
```


Body Cookies (1) Headers (14) Test Results 200 OK 100 ms 1.24 KB Save Response

{} JSON Preview Visualize


```
{
    "debug_data": {
        "account_number": "3604620891",
        "balance": 1000.0,
        "fields_registered": [
            "username",
            "password",
            "account_number"
        ],
        "is_admin": false,
        "raw_data": {
            "password": "password123",
            "username": "user2"
        },
```

Impact

- Information disclosure
- Increased attack surface

Violated Standards

- **PCI DSS Requirement 6 -** Develop and Maintain Secure Systems and Software

OWASP API Top 10

- API3: BOPLA - Excessive Data Exposure
- API7: Security Misconfiguration

Remediation

- Remove debug output from production responses
- Implement centralized error handling

Finding 2 : Virtual Card Creation Exposes Full Cardholder Data

Endpoint: *POST /api/virtual-cards/create*

Description

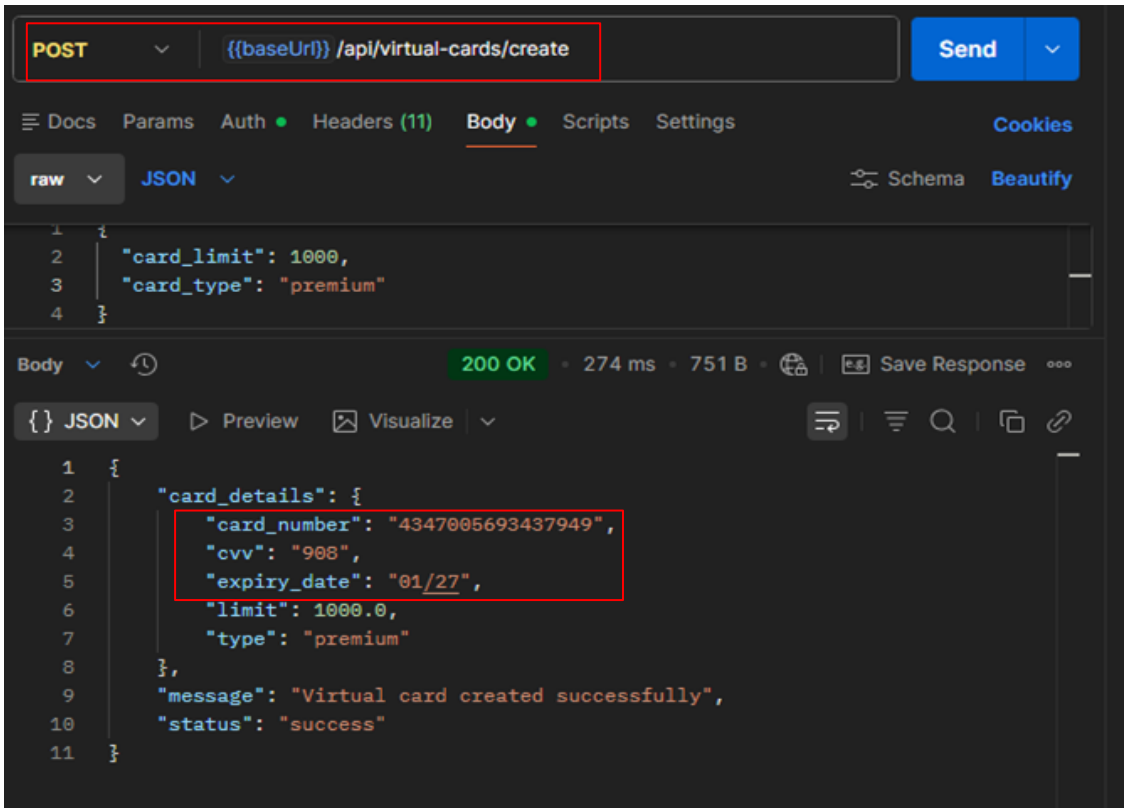
The response exposes full virtual card details, including:

- Card number (PAN)
- CVV
- Expiry date

Exposing CVV values is strictly forbidden under PCI DSS.

Evidence

I created a virtual card for each user and the response includes full card details



Impact

- Card-not-present fraud
- Full card compromise

Violated Standards

- **PCI DSS Requirement 3** – Protect stored account data
- **PCI DSS Requirement 4** – Protect Cardholder Data with strong cryptography during transmission over open, public networks

OWASP API Top 10

- API3: BOPLA - Excessive Data Exposure

Remediation

- Implement card tokenization

- Never store or return CVV values
- Mask or truncate Primary Account Number - PAN (show last 4 digits only)
- Encrypt sensitive fields at rest example using TDES/TDEA (Triple Data Encryption Algorithm) with triple-length keys

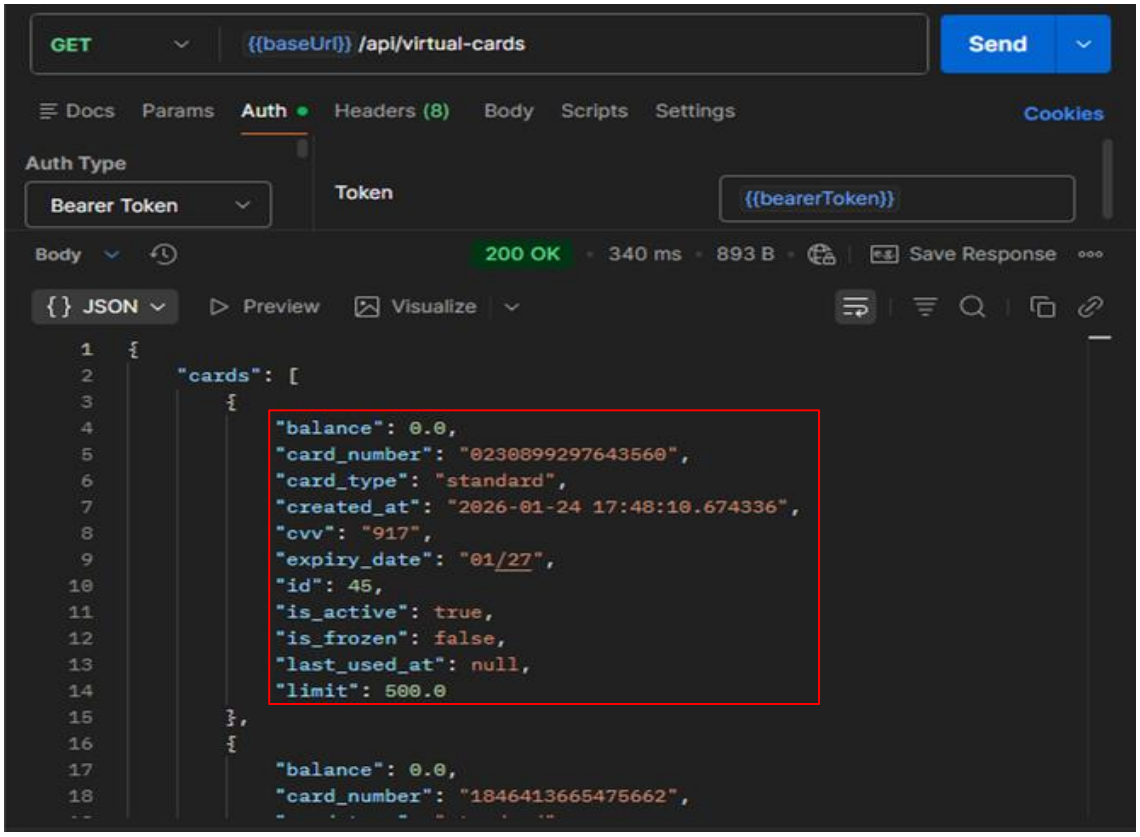
Finding 3: Virtual Cards Listing Exposes PAN and CVV

Endpoint: *GET /api/virtual-cards*

Description

This endpoint returns a list of virtual cards with full PAN and CVV values for each card.

Evidence



Impact

- Mass compromise of all user cards

Violated Standards

- **PCI DSS Requirement 3** – Protect stored cardholder data
- **PCI DSS Requirement 7** – Restrict data access

OWASP API Top 10

- API3: BOPLA - Excessive Data Exposure

Remediation

- Return only tokenized card details
- Mask PANs
- Remove CVV from all responses

Finding 4: Virtual Card Transactions Accessible via ID Manipulation

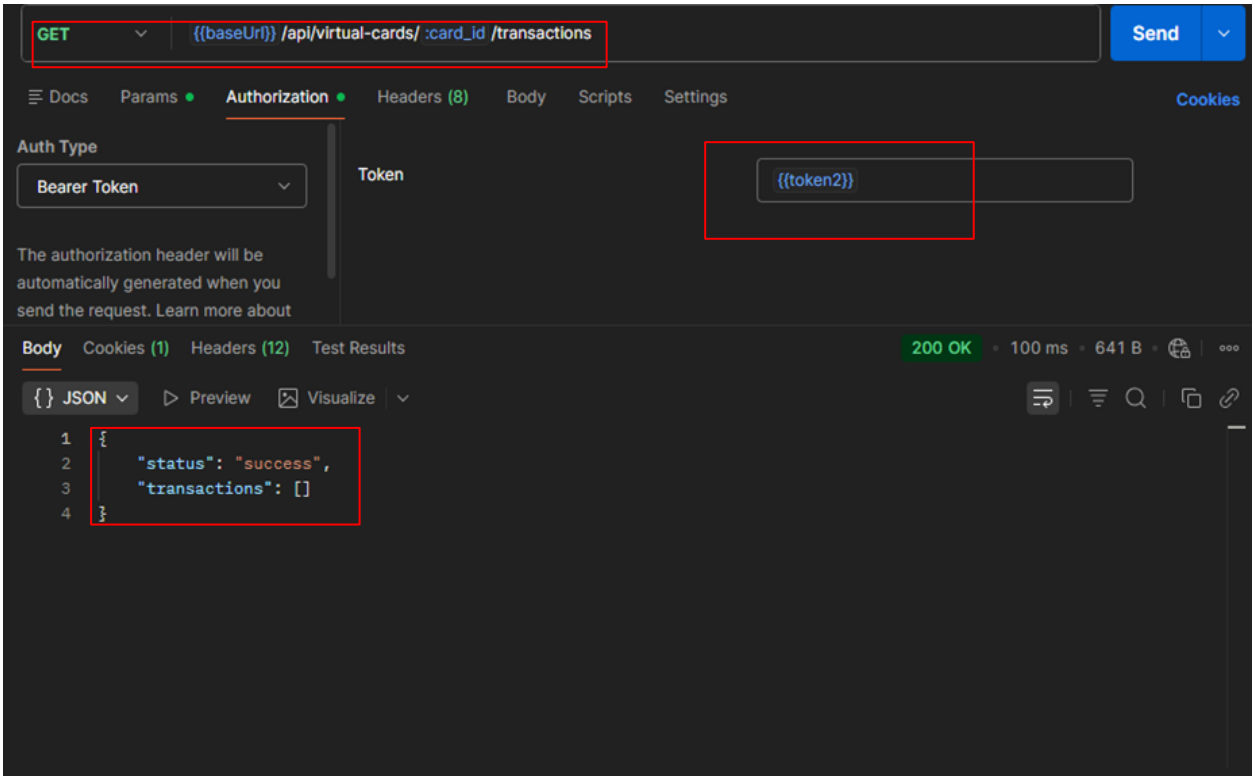
Endpoint: *GET /api/virtual-cards/{card_id}/transactions*

Description

Transaction histories can be accessed by modifying the card_id path parameter without verifying ownership.

Evidence

Whilst logged in as user2, using user2's token**(token2)** i was able to access user1's virtual card (**id: 45**) transactions



Impact

- Unauthorized access to transaction history
- Privacy violations

Violated Standards

- **PCI DSS Requirement 7** – Restrict access to card data

OWASP API Top 10

- API1: Broken Object Level Authorization (BOLA)

Remediation

- Enforce ownership validation using authenticated user identity
- Bind card_id to user session or JWT subject

Finding 5: Bill Payment History Exposes Card Numbers

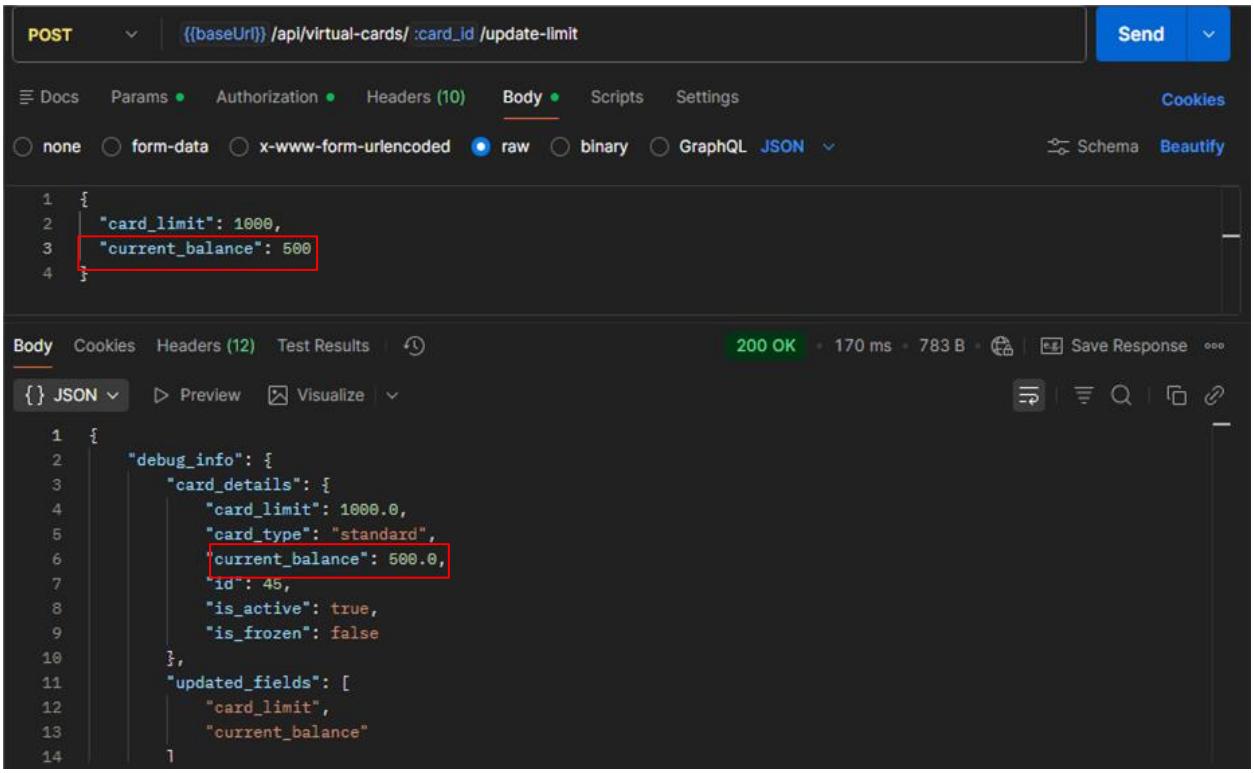
Endpoint: *GET /api/bill-payments/history*

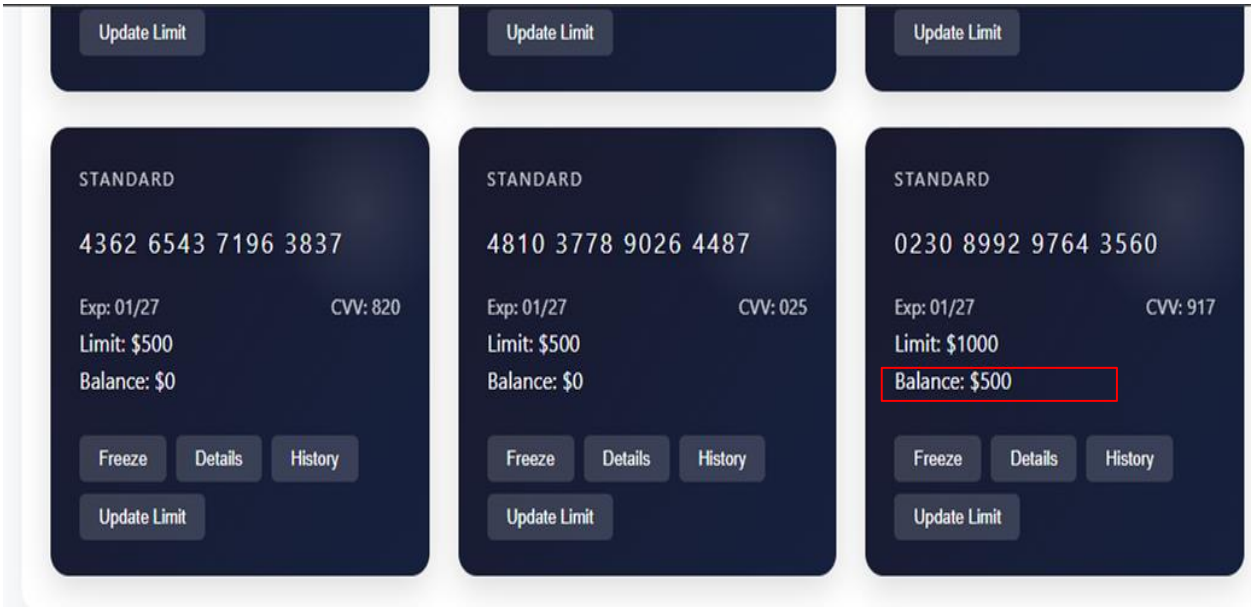
Description

Payment history responses expose full card numbers associated with past transactions.

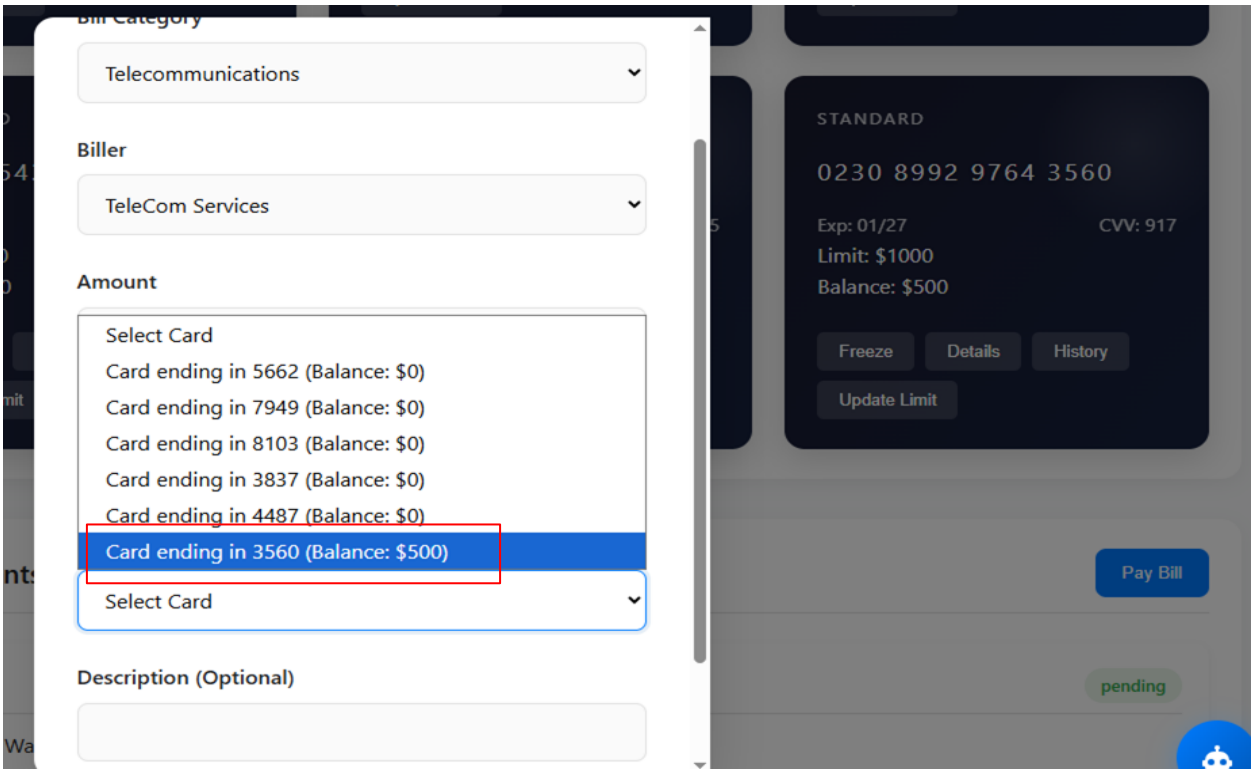
Evidence

The */api/virtual-cards/:card_id/update-limit* endpoint is vulnerable to mass assignment so I used it to load one of user1's virtual cards by adding "current_balance" to the request body.

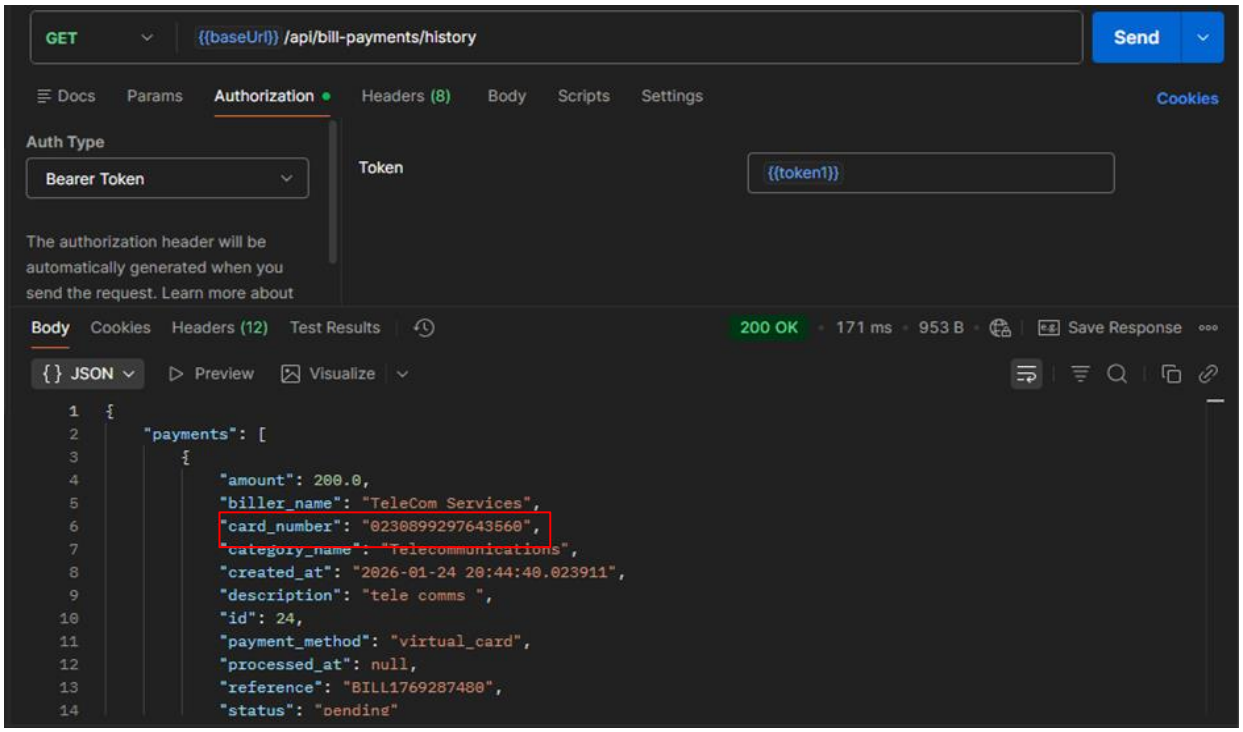




I used this card to pay for bills



when I requested bill payment history for user1, the response contained the card number.



Impact

- Historical card data leakage
- PCI DSS non-compliance

Violated Standards

- **PCI DSS Requirement 3** – Protect stored account data
- **Requirement 4** – Secure cardholder data transmission

OWASP API Top 10

- API3: BOPLA - Excessive Data Exposure

Remediation

- Mask PANs (last 4 digits only)
- Replace card numbers with secure tokens
- Remove unnecessary payment fields

Finding 6: Billers Endpoint Exposes Account Numbers

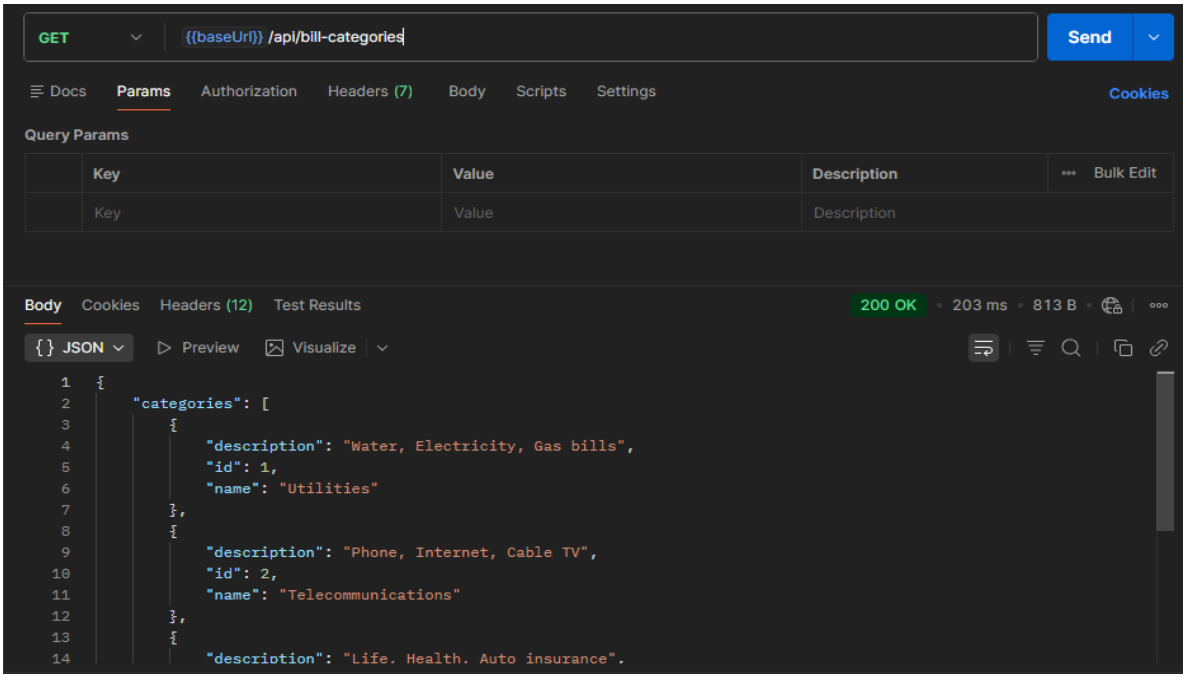
Endpoint: *GET /api/billers/by-category/{category_id}*

Description

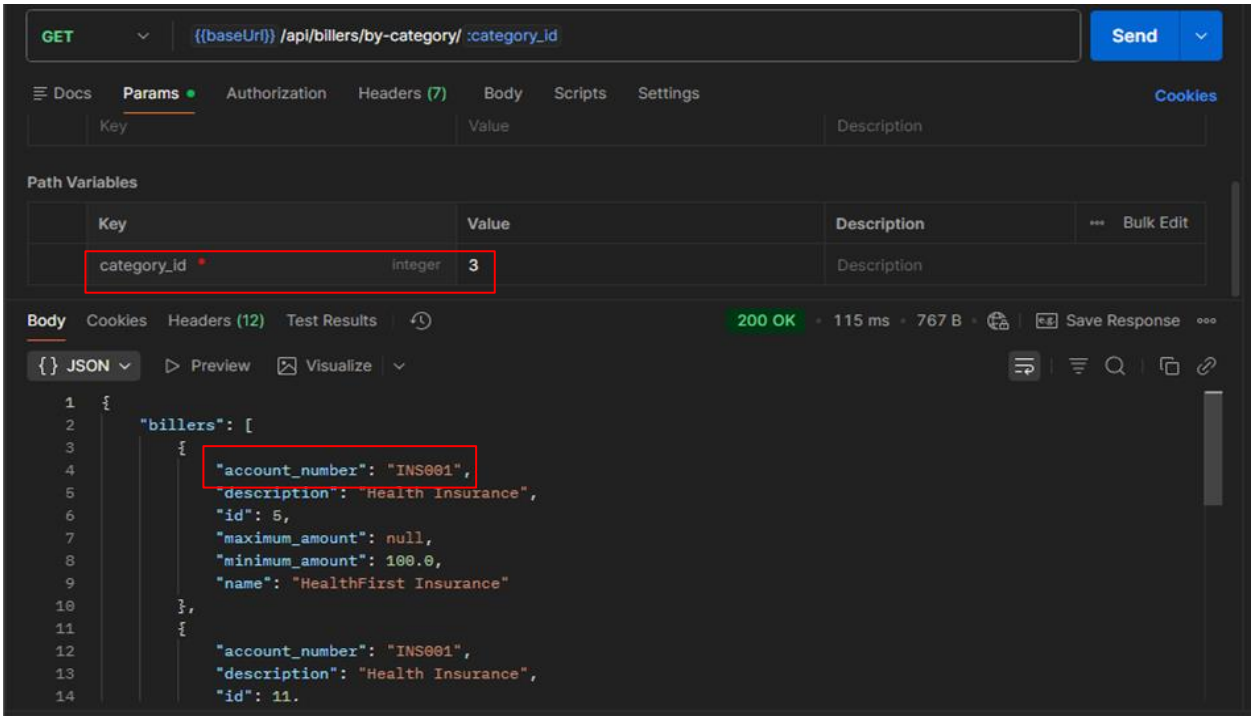
This endpoint exposes biller account numbers without sufficient protection or masking.

Evidence

I got the list of biller categories and their ids



I accessed the billers by category_id and the response includes the biller account number



Impact

- Exposure of financial identifiers
- Increased fraud and impersonation risk

Violated Standards

- **PCI DSS Requirement 3** – Protect sensitive data
- **PCI DSS Requirement 7** – Restrict data access

OWASP API Top 10

- API3: BOPLA - Excessive Data Exposure

Remediation

- Mask account numbers
- Restrict endpoint access
- Encrypt sensitive fields

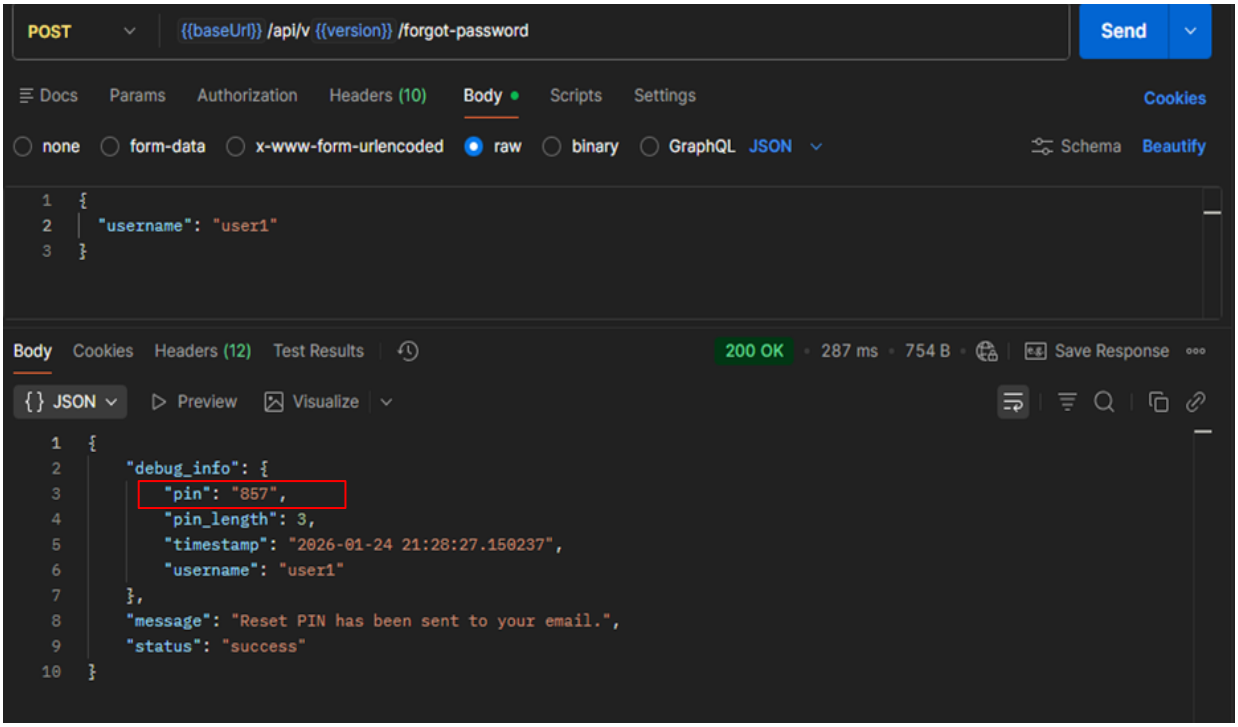
Finding 7: Password Reset Endpoint Exposes Reset Credentials

Endpoint: *POST /api/v1/forgot-password*

Description

This endpoint exposes password reset PINs directly in API responses, enabling account takeover.

Evidence



Impact

- Authentication bypass
- Unauthorized account access

Violated Standards

- **PCI DSS Requirement 8** – Identify users and authenticate access to system components

OWASP API Top 10

- API2: Broken Authentication

Remediation

- Never return reset credentials in responses
- Use one-time, time-limited tokens
- Deprecate vulnerable API versions

Finding 8: Transaction History Accessible by Account Number Manipulation

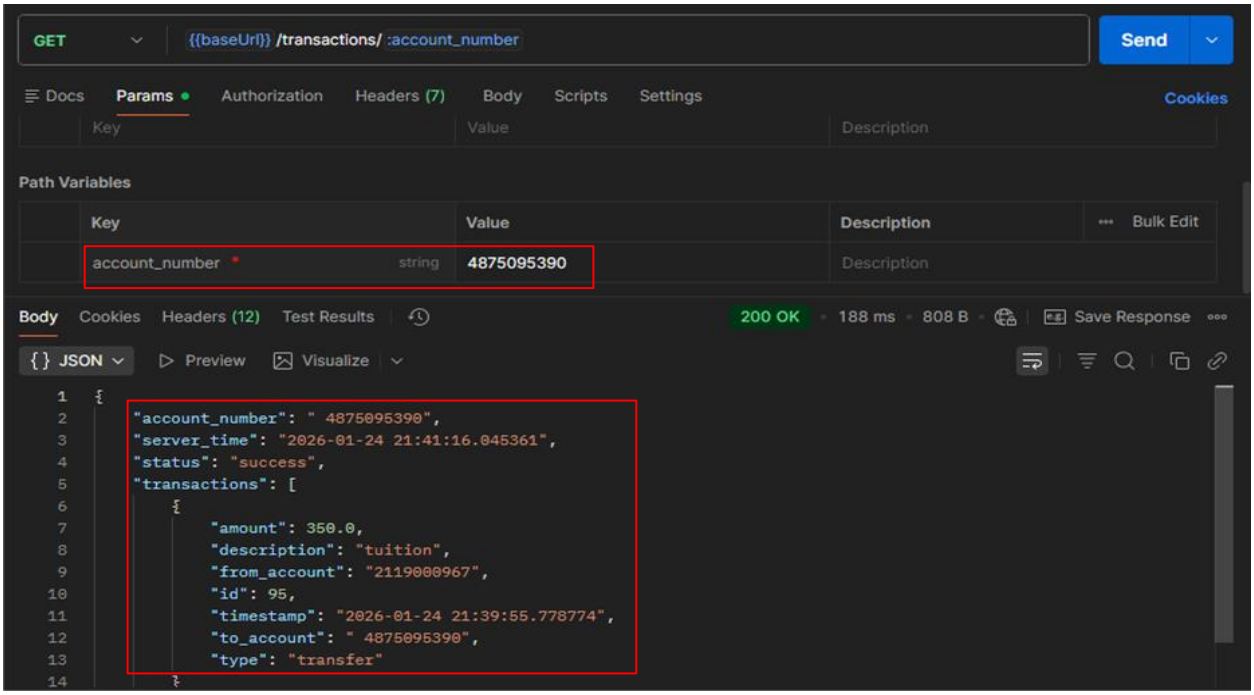
Endpoint: *GET /api/transactions/{account_number}*

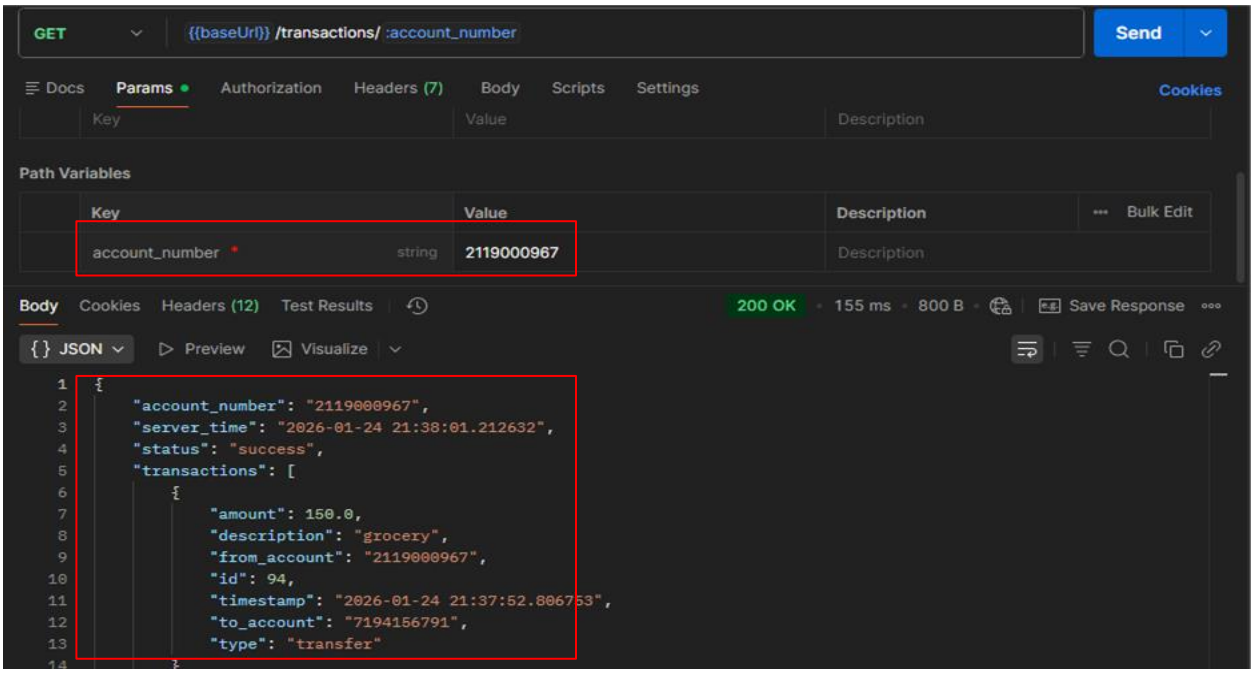
Description

Users can access transaction histories of accounts by modifying the path parameter.

Evidence

I was able to access user1 and user2's transaction history by using their account numbers without even authenticating





Impact

- Unauthorized financial data access
- Privacy breach

Violated Standards

- **PCI DSS Requirement 7** – Restrict access to cardholder data

OWASP API Top 10

- API1: Broken Object Level Authorization

Remediation

- Enforce ownership checks
- Bind account numbers to authenticated users
- Avoid exposing account numbers in URLs

Finding 9: Unauthorized Balance Disclosure via check_balance Endpoint

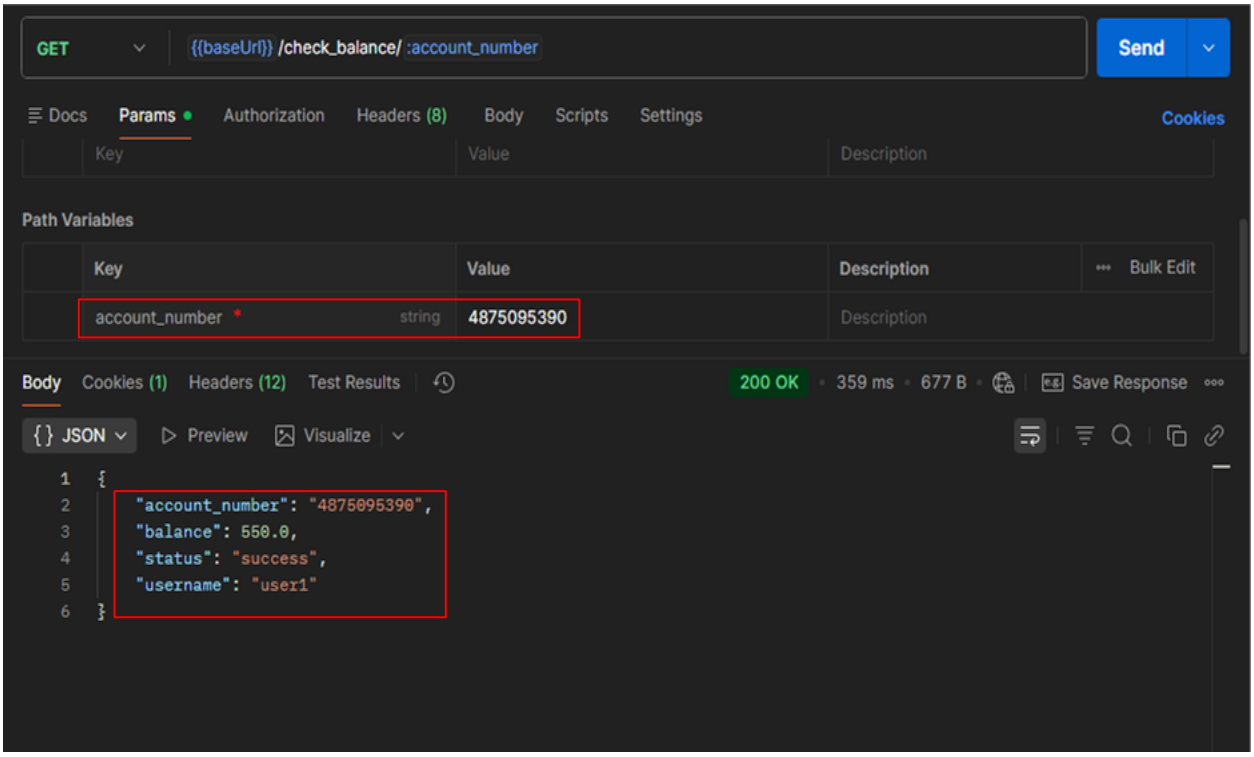
Endpoint: *GET /check_balance/{account_number}*

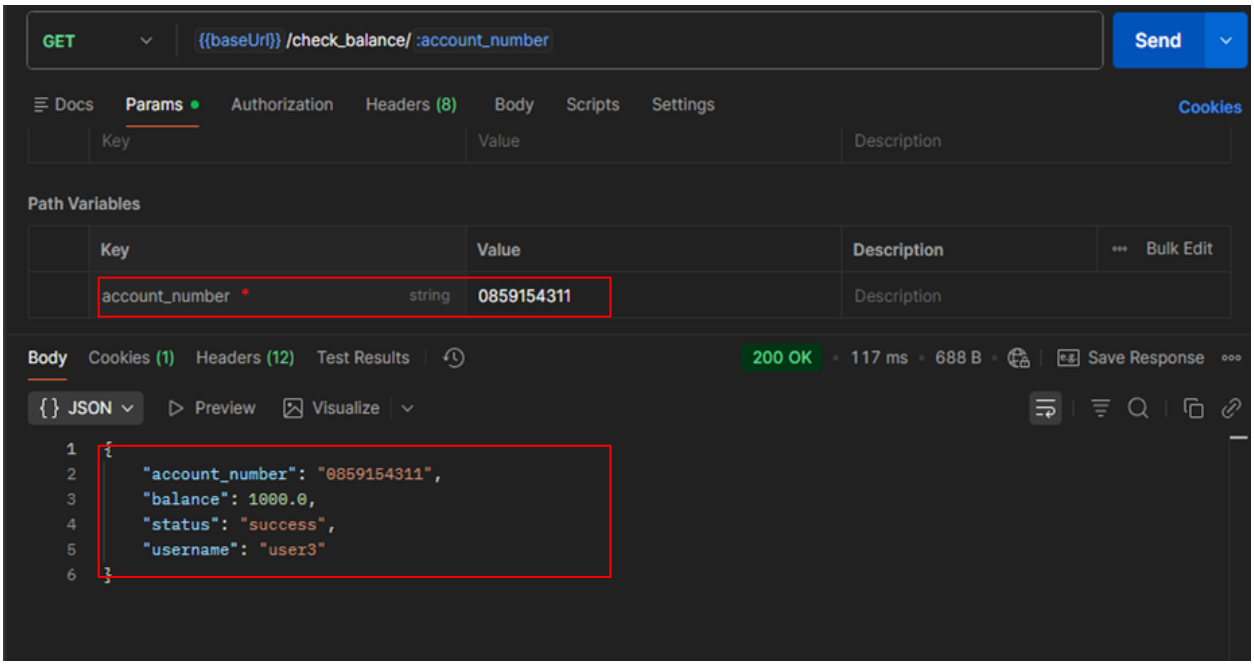
Description

The */check_balance* endpoint allows retrieval of account balances by supplying an arbitrary account number. The API does not verify if the user is authenticated and if the user owns the requested account, resulting in unauthorized disclosure of financial data.

Evidence

I was able to view 2 users' account balances by using their account numbers in the requests and these were unauthenticated requests





Impact

- Unauthorized access to account balances
- Privacy violation
- Enables targeted fraud and reconnaissance

Violated Standards

- **PCI DSS Requirement 7** – Restrict access to system data
- **PCI DSS Requirement 8** – Enforce strong authentication and authorization

OWASP API Top 10

- API1: Broken Object Level Authorization
- API3: BOPLA - Excessive Data Exposure

Remediation

- Bind account numbers to authenticated user identity

- Do not expose account numbers in URLs
- Use server-side account resolution
- Enforce authorization checks on every request

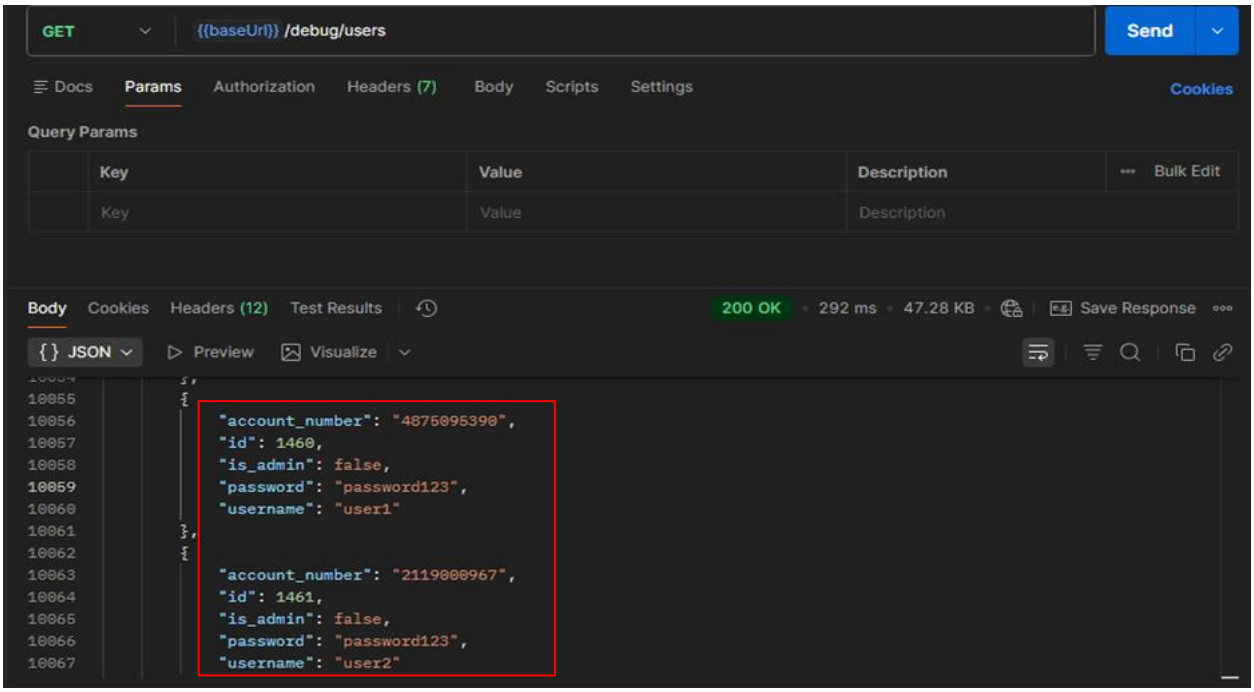
Finding 10: Exposure of Unencrypted Credentials

- **Endpoint:** *GET /debug/users*

Description

The */debug/users* endpoint is undocumented in the API spec but exposes all usernames and passwords in plaintext. This allows immediate account compromise without any brute-force effort.

Evidence



Violated Standards

PCI Requirement 8.2.1: This requirement mandates that all authentication credentials must be rendered unreadable during transmission and storage.

Remediation

- Remove all debug endpoints from production.

- Implement salted, cryptographically strong hashes for passwords.

Conclusion

The Vulnerable Bank API demonstrates so many violations of PCI DSS requirements related to cardholder data protection, authentication and access control. The exposure of PANs, CVVs, usernames, passwords and account numbers as plain text represents a critical failure to implement fundamental security controls.

By adopting **tokenization**, **encryption**, **strict authorization checks** and **secure API response design**, the application can significantly reduce its risk profile and align with PCI DSS requirements.

End of report